

When several functions in the code all operate on the same data structure, it's usually best to group them together as the methods and attributes of a class. Let's redesign this in the *tictactoe.py* program to use a `TTTBoard` class that will store the `board` dictionary in an attribute named `spaces`. The functions that had `board` as a parameter will become methods of our `TTTBoard` class and use the `self` parameter instead of a `board` parameter.

Open a new file editor window, enter the following code, and save it as *tictactoe_oop.py*:

```
# tictactoe_oop.py, an object-oriented tic-tac-toe game.

ALL_SPACES = list('123456789') # The keys for a TTT board.
X, O, BLANK = 'X', 'O', ' ' # Constants for string values.

def main():
    """Runs a game of tic-tac-toe."""
    print('Welcome to tic-tac-toe!')
    gameBoard = TTTBoard() # Create a TTT board object.
    currentPlayer, nextPlayer = X, O # X goes first, O goes next.

    while True:
        print(gameBoard.getBoardStr()) # Display the board on the screen.

        # Keep asking the player until they enter a number 1-9:
        move = None
        while not gameBoard.isValidSpace(move):
            print(f'What is {currentPlayer}\''s move? (1-9)')
            move = input()
        gameBoard.updateBoard(move, currentPlayer) # Make the move.

        # Check if the game is over:
        if gameBoard.isWinner(currentPlayer): # First check for victory.
            print(gameBoard.getBoardStr())
            print(currentPlayer + ' has won the game!')
            break
        elif gameBoard.isBoardFull(): # Next check for a tie.
            print(gameBoard.getBoardStr())
            print('The game is a tie!')
            break
        currentPlayer, nextPlayer = nextPlayer, currentPlayer # Swap turns.
    print('Thanks for playing!')

class TTTBoard:
```